

Serverless Webhook Processing with Retry, Dead-Letter Queue, and Idempotency

Cloud Computing

Sriraj Gannavaram
x23431873
Program Code - Research in Computing CA1
National College of Ireland

1 Research Problem Background

With serverless computing being an maturing paradigm, it allows developers to shift from managing the infrastructure for applications to posing logic for their applications without concern for infrastructure management Wen et al. (2023). One of the reasons serverless architectures are effectively utilized is due to being event-driven architectures that capture and process events from webhook sources such as payment gateways, CI/CD pipeline triggers, or third-party APIs Jonas et al. (2019). Unfortunately, many implementations of webhooks process them only one time without implementing sufficient retry mechanisms or corrections, creating issues where lost events are created due to downstream service errors, timeouts, and/or network issues Castro et al. (2019).

The weaknesses are more evident when evaluating the critical gap of robust failure handling within production systems. If webhooks fail, organizations do not have a means of determining which events have failed, do not have the ability to re-play them after resolution of the issue, and have a high probability for data inconsistency within their distributed system as a result of not being able to process or replay the failed events Marin et al. (2022). In addition, the stateless nature of serverless functions complicates the enforcement of idempotency when processing multiple webhook events because it can result in multiple identical webhook events being processed as separate events Barcelona-Pons et al. (2022). Consequently, the limitations presented here hinder operations that are business critical, whereby, reliable webhook delivery is necessary to establish and maintain operational continuity and consistency across all systems Shafiei et al. (2022).

2 Research Question

How can a serverless webhook processing architecture incorporating exponential backoff retry mechanisms, dead-letter queues, and idempotency controls improve event delivery reliability and reduce data inconsistencies compared to traditional once-only processing approaches in cloud environments?

3 Justification

The research topic outlined above has not been fully addressed in existing literature on Serverless Computing; specifically, mechanisms for retrying failed webhook deliveries.

There are several cloud service providers (e.g., AWS, Microsoft Azure, Google Cloud) that provide native cloud technologies to build and implement the proposed solution within the 12-week timeframe. All of the cloud service providers mentioned above have cloud-native serverless technologies (e.g., AWS Lambda, Azure Functions, and Google Cloud Functions) and cloud-native managed queue technologies (e.g., Amazon Simple Queue Service [SQS], Azure Queue Storage, and Google Cloud Pub/Sub).

The tools available to implement the proposed solutions are all readily available through the cloud providers mentioned above, (SDKs from each of the cloud providers), infrastructure as code (Terraform), and integrated development environments (IDEs). Thus, building, implementing, and validating the solution within the 12-week timeframe will be a challenge, however, it is feasible to complete the requested work in this timeframe.

The research will provide quantifiable metrics that can be measured. These metrics include the percentage of successfully processed webhooks, the number of retry attempts required for webhooks, the number of failed webhook delivery attempts which are later successful due to utilization of a dead-letter queue (DLQ), the effectiveness of duplicate prevention, the processing latency of webhooks, and cost savings related to processing webhooks.

Organizations are increasingly relying on webhooks for important elements of their business such as: payment confirmations, order fulfillment, deployment automation, and real-time notifications (Li, 2022).

Failures in processing webhooks can lead to the loss of revenue, customer dissatisfaction, violations of regulations, and disruption of operations (Jonas et al., 2019).

This research provides very practical contributions to the body of knowledge by providing a robust solution to manage intelligent retry logic, to fail in isolation through dead letter queues and to prevent duplicates through idempotency keys which can be used by production systems. The contribution is notable because through the use of this solution, organizations processing large volumes of webhooks daily can prevent revenue losses, improve the overall reliability of their systems and reduce the overall operational overheads associated with processing. Additionally, this research is ethical in that it does not involve human subjects in the design and testing of systems, uses publicly available software tools and is intended to improve the reliability of technical systems which will benefit the end user through improved service quality provided to them.

4 Specific Items to Be Addressed

- Item #1 **Webhook Receiver and Queue Integration:** Build and deploy an HTTP serverless endpoint that is able to receive webhook HTTP POST requests, validate the authenticity of incoming webhook requests by performing HMAC-SHA256 checking (to avoid receiving spoofed requests), take idempotency keys from the request headers or request body, validate the incoming payload schemas, and store verified, incoming webhook requests to a durable message queue and respond with an immediate 2xx acknowledgement (to ensure that the provider's webhook times out quickly).
- Item #2 **Retry Logic with Exponential Backoff:** Implement a queue consumer which can process webhook events based upon user-configurable retry policies including number of retries (3 – 5 retries), exponential back-off with jitter for each retry attempt (e.g. 1 second, 2 seconds, 4 seconds, 8 seconds) in order to avoid thundering herd issues; classify failures intelligently distinguishing between transient failure conditions (e.g. timeouts, rate limits) and permanent failure conditions (e.g. authentication failure, schema validation failure) and route to a dead-letter queue automatically after exhausting the maximum number of retry attempts.
- Item #3 **Dead-Letter Queue and Replay Mechanism:** Develop an individual dead-letter queue for all events that exceed the maximum number of retries, and develop an administrator user interface that allows for viewing information about failed events' SHA-256 payload hashes (to protect the sensitive nature of this information). In addition, develop a detailed failure logging system that provides timestamps and error category codes, and implement a secure replay mechanism that allows an authorized operator to reprocess dead-lettered events after the resolution of issues that caused them to be dead-lettered while maintaining an audit trail.
- Item #4 **Idempotency and Observability:** Implementing persistence storage of processed event identifiers is a method of enforcing (idempotency) the configuration time windows for (TTL) between 24-72 hours. Using (distributed caching) to center fast idempotency checks by providing the necessary information (metrics) to see received count, processed count, retry count, dead letter count, duplicate prevention count, and processing latency percentiles (P50, P95, P99). In addition, developing a (real-time dashboard) for system health visibility and to generate alerts when anomalies occur.

Item #5 **Security and Encryption:** The queue contents, and the contents of dead-letter queues shall be encrypted at rest. The associated encryption shall be carried out using cloud-native encryption services (e.g., AWS Key Management Service (KMS) or Azure Key Vault). The organization also requires the implementation of webhook signature verification using provider-specific signature verification algorithms. Finally, the organization will enforce the least-privileged method of access control using identity and access management policies that restrict the operations of queues to only those services that have been authorized. The organization will maintain comprehensive audit logs of all activities relating to receipt and processing of webhooks. The organization will also ensure compliance with all applicable data protection regulations through proper data handling and retention policies.

References

- Barcelona-Pons, D., Sutra, P., Sánchez-Artigas, M., París, G. & García-López, P. (2022), ‘Stateful serverless computing with crucial’, *ACM Transactions on Software Engineering and Methodology* **31**(3), 1–38.
- Castro, P., Ishakian, V., Muthusamy, V. & Slominski, A. (2019), ‘The rise of serverless computing’, *Communications of the ACM* **62**(12), 44–54.
- Jonas, E., Schleier-Smith, J., Sreekanti, V., Tsai, C.-C., Khandelwal, A., Pu, Q., Shankar, V., Carreira, J., Krauth, K., Yadwadkar, N., Gonzalez, J. E., Popa, R. A., Stoica, I. & Patterson, D. A. (2019), Cloud programming simplified: A berkeley view on serverless computing, Technical Report UCB/EECS-2019-3, EECS Department, University of California, Berkeley.
- Marin, E., Perino, D. & Di Pietro, R. (2022), ‘Serverless computing: a security perspective’, *Journal of Cloud Computing* **11**(1), 1–30.
- Shafiei, H., Khonsari, A. & Mousavi, P. (2022), ‘Serverless computing: A survey of opportunities, challenges, and applications’, *ACM Computing Surveys* **54**(11s), 1–32.
- Wen, J., Chen, Z., Jin, X. & Liu, X. (2023), ‘Rise of the planet of serverless computing: A systematic review’, *ACM Transactions on Software Engineering and Methodology* **32**(5), 1–61.